Day 84: Byte-Pair Encoding, From Scratch, on Real Text

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 1 | Complete study guide

Watch: Byte Pair Encoding Tokenization (Hugging Face) | Let's build the GPT Tokenizer (Andrej Karpathy)

Script: `day84_byte_pair_encoding.py` · every number in this guide is copied by program from `day84_run_output.txt`, the real output of that script.

Every model since Day 61 has read whole words from a fixed vocabulary, and every word outside it became <unk>: 20.2% of the test words in Day 62's tagger. Byte-pair encoding learns a vocabulary of subwords by repeatedly merging the most frequent adjacent pair of symbols. Trained on Week 9's real sentences it encodes every test sentence losslessly, needs 1.454 tokens per word with 4071 symbols, and spells Day 62's unknown words out of known pieces. What it cannot handle is a character it never saw: four such characters occur in the test text, which is why byte-level BPE starts from the 256 byte values instead.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

A **tokenizer** turns text into a sequence of **tokens** from a fixed vocabulary, and back. **Byte-pair encoding (BPE)** starts from single characters and learns **merges**: each round it finds the adjacent pair of symbols that occurs most often in the training text and makes it one new symbol. To tokenize a new word, it applies the learned merges in the order they were learned.

REAL-WORLD SCENARIO

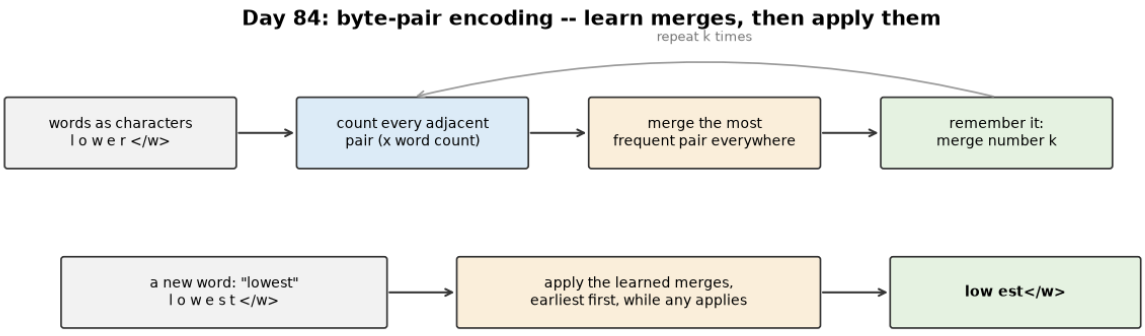
Shorthand: a note-taker who writes 'th' as one stroke because it appears in nearly every sentence, then 'the', and eventually common whole words, while still spelling out rare names letter by letter.

MATHEMATICS

repeat k times: (a, b) = argmax over adjacent pairs of count(a, b); replace every a b by the new symbol ab

vocabulary = characters + </w> + k merged symbols; a word is encoded by applying merges 1, 2, ..., k in order

SYSTEM DIAGRAM



Learning merges, then applying them to a new word.

Week 13 is built part by part

Days 84–89 each add a short exercise script, one part of a tiny GPT-style model built on Day 90. Today's part is the tokenizer (`day84_exercise.py`).

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. BPE by hand

DEFINITION — start here, no prior knowledge assumed

Sennrich et al.'s four-word example: low × 5, lower × 2, newest × 6, widest × 3, ten merges.

REAL-WORLD SCENARIO

Watching the note-taker invent their first ten shorthand strokes.

IMPLEMENTATION

```
def pair_counts(vocab):
    counts = Counter()
    for symbols, freq in vocab.items():
        for a, b in zip(symbols, symbols[1:]):
            counts[a, b] += freq
    return counts
def merge_word(symbols, pair, new):
    out, i = [], 0
    while i < len(symbols):
        if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
            out.append(new); i += 2
        else:
            out.append(symbols[i]); i += 1
    return tuple(out)
def train_bpe(word_counts, n_merges):
    vocab = {tuple(w) + (EOW,): c for w, c in word_counts.items()}
    merges = []
    for _ in range(n_merges):
        counts = pair_counts(vocab)
        if not counts:
            break
        best = max(counts, key=lambda p: (counts[p], p))
        new = best[0] + best[1]
        vocab = {merge_word(s, best, new): c for s, c in vocab.items()}
        merges.append((best, counts[best]))
    return merges
```

ACTUAL OUTPUT

```
 round  merge            count   words afterwards
     1  t + </w>             9   l o w </w>  l o w e r </w>  n e w e s t</w>  w i d e s t</w>
     2  s + t</w>            9   l o w </w>  l o w e r </w>  n e w e st</w>  w i d e st</w>
     3  e + st</w>           9   l o w </w>  l o w e r </w>  n e w est</w>  w i d est</w>
     4  o + w                7   l ow </w>  l ow e r </w>  n e w est</w>  w i d est</w>
     5  l + ow               7   low </w>  low e r </w>  n e w est</w>  w i d est</w>
     6  w + est</w>          6   low </w>  low e r </w>  n e west</w>  w i d est</w>
     7  n + e                6   low </w>  low e r </w>  ne west</w>  w i d est</w>
     8  ne + west</w>        6   low </w>  low e r </w>  newest</w>  w i d est</w>
     9  low + </w>           5   low</w>  low e r </w>  newest</w>  w i d est</w>
    10  w + i                3   low</w>  low e r </w>  newest</w>  wi d est</w>
encoding words never seen in this corpus with the 10 merges:
  lowest   -> low est</w>
  newer    -> ne w e r </w>
  wider    -> wi d e r </w>
Actual output from this lesson's run.
```

What it means

The first merges join the endings shared by newest and widest (t, st, est), then 'low'. The end-of-word mark keeps 'est' at the end of a word apart from 'est' inside one. New words reuse the pieces: 'lowest' becomes low + est. 'newer' does not become 'new + er': the merges that would build those pieces were not among the ten most frequent.

WHY CHOOSE THIS?

Simple, and the merges are readable.

WHY NOT THIS? (TRADEOFFS)

Greedy: each merge is the most frequent pair now, with no look-ahead.

2. Training on real text

DEFINITION — start here, no prior knowledge assumed

Week 9's 2,001 training sentences, lowercased: 25148 words. 4000 merges.

REAL-WORLD SCENARIO

The note-taker on a real term of lectures.

ACTUAL OUTPUT

```
25148 words (4812 distinct) in the 2,001 training sentences, lowercased; 70 different characters,
    so 71 starting symbols with </w>.
the first 15 merges (pair, and how often it occurred when merged):
  e+</w> (4057)   s+</w> (2415)   t+</w> (2387)   t+h (2277)   d+</w> (1890)
  i+n (1714)   a+n (1540)   e+r (1450)   y+</w> (1337)   .+</w> (1282)
  o+n (1097)   o+</w> (1031)   o+u (1007)   th+e</w> (983)   o+r (901)
merges 1000, 2000, 4000: de+partment</w> (11), ap+pe (5), la+unched</w> (2)
4000 merges took 42.8 s here (every pair recounted every round)
Actual output from this lesson's run.
```

What it means

The first merges are word endings ('e</w>' 4057 times) and the commonest pairs of English ('th', 'in', 'an', 'er'); merge 14 already makes 'the</w>'. By merge 1000 a merged pair occurs only 11 times, and by 4000 only 2: each new symbol covers less text. Recounting every pair every round took 42.8 s for 4000 merges here; real tokenizers update the counts incrementally.

WHY CHOOSE THIS?

Learned from the data, no linguistics needed.

WHY NOT THIS? (TRADEOFFS)

This simple version recounts everything each round: slow on large text.

3. Encoding is lossless

DEFINITION — start here, no prior knowledge assumed

Every test sentence encoded with all 4000 merges and decoded again.

REAL-WORLD SCENARIO

Shorthand must read back to exactly the words that were said.

IMPLEMENTATION

```
def encode_word(word, ranks):
    symbols = tuple(word) + (EOW,)
    while len(symbols) > 1:
        pairs = [(ranks.get(p, np.inf), p) for p in zip(symbols, symbols[1:])]
        rank, best = min(pairs)
        if rank == np.inf:
            break
        symbols = merge_word(symbols, best, best[0] + best[1])
    return symbols
def decode(symbols):
    return "".join(symbols).replace(EOW, " ").strip()
```

ACTUAL OUTPUT

```
sentences that do not decode back exactly: 0 of 1647
tokens: 26842 for 18460 words (1.454 per word)
example: i was wondering if it would be ok for me to sit in with you one day to get a general feel
    of your end of the process .
encoded: i</w> | was</w> | wondering</w> | if</w> | it</w> | would</w> | be</w> | ok</w> | for</w>
    | me</w> | to</w> | s it</w> | in</w> | with</w> | you</w> | one</w> | day</w> | to</w> |
    get</w> | a</w> | general</w> | feel</w> | of</w> | your</w> | end</w> | of</w> | the</w> |
    process</w> | .</w>
Actual output from this lesson's run.
```

What it means

All 1,647 sentences come back exactly (0 of 1647). In the example, every word is one token except 'sit', split as 's it' because 'sit' is rare in the training text.

WHY CHOOSE THIS?

No information is lost.

WHY NOT THIS? (TRADEOFFS)

Lossless only for characters seen in training (Section 5).

4. Vocabulary size against sequence length

DEFINITION — start here, no prior knowledge assumed

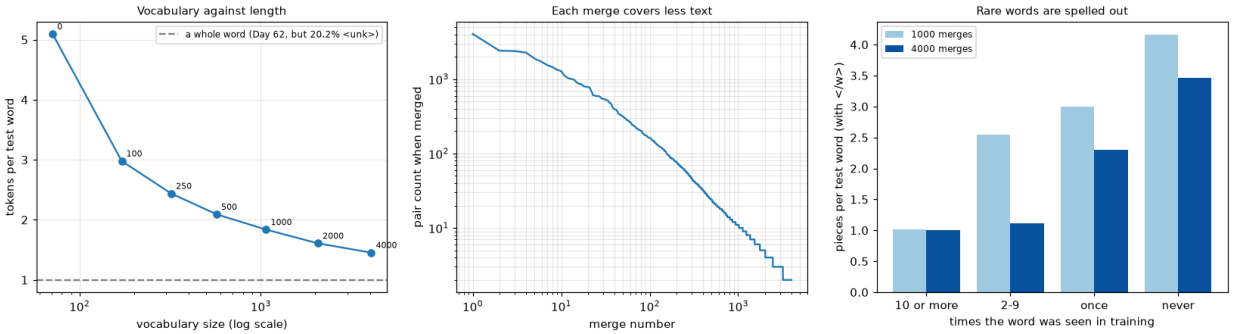
The first k merges for several k: the vocabulary size, the tokens per test word, and the share of test words that become one token.

REAL-WORLD SCENARIO

More shorthand strokes to learn, fewer strokes per word written.

ACTUAL OUTPUT

```
 merges  vocabulary  tokens per word  one-token words
      0          71            5.096            0.000
    100         171            2.979            0.327
    250         321            2.437            0.466
    500         571            2.092            0.558
   1000        1071            1.837            0.637
   2000        2071            1.609            0.722
   4000        4071            1.454            0.788
(with 0 merges every character is a token, and </w> is its own token: a 3-letter word is 4 tokens)
Actual output from this lesson's run.
```



Left: tokens per test word against vocabulary size. Middle: how often each merged pair occurred, by merge number (both axes log scale). Right: pieces per test word by how often the word was seen in training.

What it means

Characters alone take 5.096 tokens per word (the end mark included). 1000 merges bring it to 1.837, 4000 to 1.454, with 0.788 of the test words a single token. Day 62's word vocabulary needs one token per word with 2,082 entries but cannot read a fifth of the words. Longer sequences cost attention n squared (Day 81); a bigger vocabulary costs a larger embedding table and output layer.

WHY CHOOSE THIS?

One knob, the number of merges, trades vocabulary for length.

WHY NOT THIS? (TRADEOFFS)

The right size depends on the model and the data.

5. No more <unk> — and the characters BPE still cannot read

DEFINITION — start here, no prior knowledge assumed

The test words outside Day 62's vocabulary, how many pieces BPE needs for them, and the test characters never seen in training.

REAL-WORLD SCENARIO

The shorthand can spell any new name it has the letters for, but not a word in a different alphabet.

ACTUAL OUTPUT

```
test words outside Day 62's vocabulary: 3725 of 18460 (20.2%)
characters in the test text never seen in training: 4
  U+005E: 1 byte(s) in UTF-8; in 2 test word(s)
  U+00B4: 2 byte(s) in UTF-8; in 1 test word(s)
  U+03C5: 2 byte(s) in UTF-8; in 1 test word(s)
  U+2014: 3 byte(s) in UTF-8; in 2 test word(s)
test words with such a character: 6 -- BPE keeps the character as a symbol of its own, which a
    model trained on this vocabulary has never seen. A byte-level BPE (as in GPT-2) starts from
    the 256 possible byte values instead, so no input can ever fall outside it.
BPE pieces per Day-62-unknown word: mean 3.16; one piece 0.031
examples:
  !!!!!!!!!!!!!!!!!! -> !!!!!!!! !!!!!!! !!!</w>
  backdrop         -> back dro p</w>
  concern          -> concer n</w>
  equistar         -> e qu i star</w>
  http://www.bbc.co.uk/dna/actionnetwork/c55153 -> http://www. bbc.co.uk/ d na / ac tion ne t wor
      k/ c 5 5 15 3</w>
  messages         -> mess ages</w>
  ragnarok         -> ra gn ar ok</w>
  starting         -> star ting</w>
Actual output from this lesson's run.
```

What it means

The 3725 test words Day 62 turned into <unk> get 3.16 pieces on average: nothing is unknown. The pieces are frequency-driven, not linguistic ('concer n', 'star ting'). But 4 characters in the test text never occur in training, in 6 test words, and BPE has no symbol for them. Byte-level BPE (GPT-2) avoids this by starting from bytes: every character is one to four of the 256 byte values, all in the vocabulary from the start.

WHY CHOOSE THIS?

Open vocabulary for any text in the training alphabet.

WHY NOT THIS? (TRADEOFFS)

A character-level start leaves new characters unreadable.

6. Frequent words are one piece, rare words several

DEFINITION — start here, no prior knowledge assumed

Pieces per test word, grouped by how often the word was seen in training.

REAL-WORLD SCENARIO

Daily words get one stroke; once-a-year words are written out.

ACTUAL OUTPUT

```
seen in training   test words  pieces, 1000 merges  pieces, 4000 merges
10 or more             11770                 1.02                 1.00
2-9                     2965                 2.54                 1.11
once                     955                 3.00                 2.30
never                   2770                 4.16                 3.46
Actual output from this lesson's run.
```

What it means

Words seen 10 or more times are one piece (1.00 with all merges); words never seen take 3.46. The model's sequence is short where text is common and longer only where it must be.

WHY CHOOSE THIS?

Spends the vocabulary where the text is.

WHY NOT THIS? (TRADEOFFS)

Rare words are long sequences of pieces the model must still learn to combine.

Interview preparation — technical questions

Q: Describe BPE training.

A: Start from characters; repeatedly merge the most frequent adjacent pair everywhere; remember each merge.

Q: How is a new word encoded?

A: Apply the learned merges in the order learned, while any applies.

Q: Why the end-of-word mark?

A: So that a suffix at the end of a word and the same letters inside a word stay different symbols.

Q: What did 4000 merges give here?

A: 4071 symbols, 1.454 tokens per test word, 0.788 of words one token, no <unk> words.

Q: What can character-level BPE not encode?

A: A character never seen in training: 4 here, in 6 test words.

Interview preparation — theoretical questions

Q: Why do LLMs use subword tokenizers?

A: No unknown words, and common text stays short; whole-word vocabularies cannot read new words.

Q: Why byte-level BPE?

A: Every possible input is a sequence of bytes, so nothing can fall outside the vocabulary.

Q: Are BPE pieces morphemes?

A: No: they follow frequency ('star ting'), not meaning.

Q: What does a larger vocabulary cost?

A: A bigger embedding table and output layer, and rarer tokens with less training signal each.

Key takeaway and what's next

KEY TAKEAWAY

BPE learns a vocabulary of subwords by merging the most frequent pairs. On real text it is lossless, leaves no word unknown, keeps common words as single tokens and spells rare ones out; its one gap, characters never seen in training, is what byte-level BPE closes.

- Test sentences that do not decode exactly: 0 of 1647.
- Tokens per word: 5.096 (characters) to 1.454 (4000 merges, 4071 symbols).
- Day 62's unknown words (20.2%): 3.16 BPE pieces each, none unknown.
- 4 unseen characters in the test text, in 6 words.

Suggested watch order, if starting from zero

- 1. Byte Pair Encoding Tokenization (Hugging Face) — BPE in five minutes.
- 2. Let's build the GPT Tokenizer (Andrej Karpathy) — building a GPT tokenizer, byte-level, in code.

Day 85 preview

- **Word embeddings.** A tokenizer gives ids; a model needs vectors. One-hot vectors against dense, learned embeddings, on a real vocabulary.
- Today's exercise script, `day84_exercise.py`, is a BPE tokenizer in four functions on Day 61's reviews. It works on whole text, spaces included, so its merges cross word boundaries ('the food was' becomes one token); GPT-2 splits text into words first to prevent that.